

Soutenance de thèse

Conception d'un système mémoire pour traitement de données creuses

Thèse soutenue par

Valentin ISAAC--CHASSANDE

le 11 mai 2026

Université Grenoble Alpes, CEA List, TiMA

Plan

1. Prérequis et Problématique

2. Approche 1 : Cache de réduction

3. Approche 2 : Partitionnement du cache

4. SpDCache : Fonctionnement et Architecture Matérielle

5. Evaluations et Performances de SpDCache

Plan

1. Prérequis et Problématique

2. Approche 1 : Cache de réduction

3. Approche 2 : Partitionnement du cache

4. SpDCache : Fonctionnement et Architecture Matérielle

5. Evaluations et Performances de SpDCache

Les matrices creuses

Slide 1 de HiPEAC

Les matrices creuses jusqu'au problème dans cache, et intro SpDCache

Premières slides de ASAP

Les modèles développés pour étudier et évaluer SpDCache

Trois modèles :

	C	Py	RTL
Modèle	Modèle analytique	Modèle de simulation	Implémentation RTL
Langage	C	Python	SystemVerilog
Entrées	Distributions de densités	Matrices réelles et synthétiques	Matrices réelles et synthétiques
Niveau de paramétrisation	Très élevé +++	Élevé ++	Moyen +
Vitesse de simulation	$\sim 10^5$ nnz/s +++	$\sim 10^3$ nnz/s ++	~ 10 nnz/s +
Précision	Probabilité +	Transaction ++	Cycle +++
Design	Approximatif	Exact	Exact avec contraintes matérielles

Problème :

Comment traiter efficacement des réductions irrégulières ?

Deux approches complémentaires :

Approche 1 : Cache de réduction pour calculer les réductions dans le cache et éviter des transactions inutiles

Approche 2 : Traiter l'irrégularité en s'appuyant sur la structure des matrices, partitionnement du cache

Plan

1. Prérequis et Problématique

2. Approche 1 : Cache de réduction

- Cache générique
- Cache de réduction
- Conclusion

3. Approche 2 : Partitionnement du cache

4. SpDCache : Fonctionnement et Architecture Matérielle

5. Evaluations et Performances de SpDCache

Contexte

Noyau de calcul : SpMV (multiplication matrice creuse-vecteur)

Algorithme : OP (*Outer-Product*)

Avec OP SpMV, les accès « aléatoires » se produisent sur le vecteur de sortie et sont des réductions

➔ Réductions « aléatoires »

Une réduction sur un vecteur v :

$v_{indice} += valeur$

- Une lecture en mémoire de v_{indice}
- Une accumulation pour mettre à jour v_{indice} avec $valeur$
- Une écriture en mémoire de v_{indice} mis à jour

Cette opération est noté RED :

$RED(indice, valeur)$

SpMV : $A \times b = c$

Avec A une matrice creuse de taille (M, N) , b et c des vecteurs denses de taille N et M

OP (*Outer-Product*)

for $n \in [0, N - 1]$

for $m \in [0, M - 1]$

$c_m += A_{m,n} \times b_n$

end

end

➔ Accès séquentiels sur les entrées A et b

➔ Accès « aléatoire » sur le vecteur de sortie c

Cache générique

Fonctionnement & Réduction dans un cache générique

Granularité : ligne de cache (64 octets)

Transactions mémoires : lecture et écriture de ligne de 64 octets

Réduction : $c_{indice} += valeur$

- lecture dans le cache
 - avec hit, miss ou évincement
- accumulation dans le processeur
- écriture dans le cache



<schéma d'une réduction dans un cache générique>

Cache de réduction

Qu'est-ce ?

Même caractéristiques que cache : set-associatif, plusieurs voies, lignes de cache

Traite des instructions de réduction RED

Accumulation sur place, ajout d'un accumulateur

Style write-back

Figure comme SpDCache (voir ASAP) ou on montre ce qu'il y a en plus

Cache de réduction

Une réduction dans un cache de réduction



list

<schéma d'une réduction dans un cache
générique>



list

<schéma d'une réduction dans un cache de
réduction>

Comparaison du nombre d'instruction / le processeur n'attend pas les REDs

Perf

Python 25/75

Conclusion

Notre cache doit supporter les réductions

Note : existant, PHI, SortCache, ASA le font déjà

Plan

1. Prérequis et Problématique

2. Approche 1 : Cache de réduction

3. Approche 2 : Partitionnement du cache

4. SpDCache : Fonctionnement et Architecture Matérielle

5. Evaluations et Performances de SpDCache

Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖



list

Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖

Petite matrice



Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖

Petite matrice, bande horizontale :



Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖

Petite matrice, bande horizontale :

- Tiens dans le cache, pas d'évincement
- Le cache est utile ✔
- Cas peu intéressants ✖



Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖

Petite matrice, bande horizontale :

- Tiens dans le cache, pas d'évincement
- Le cache est utile ✔
- Cas peu intéressants ✖

Matrice bandée :

- Évincements répétés dans le cache



Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- ➔ Le cache n'est pas utile ✖

Petite matrice, bande horizontale :

- Tiens dans le cache, pas d'évincement
- ➔ Le cache est utile ✔
- ➔ Cas peu intéressants ✖

Matrice bandée :

- Évincements répétés dans le cache
- ➔ Le cache n'est pas utile ✖
- ➔ Structure très courante ✔



Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖

Petite matrice, bande horizontale :

- Tiens dans le cache, pas d'évincement
- Le cache est utile ✔
- Cas peu intéressants ✖

Matrice bandée :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖
- Structure très courante ✔

Matrice parfaitement bandée :

- Nombre d'évincements limité
- Le cache est utile ✔
- Structure peu courante ✖



Traffic mémoire théorique avec des matrices bandées

c

Traffic mémoire du vecteur de sortie

versus

densité en dehors de la bande,
pour différentes tailles de cache

- Densité dans la bande fixe (10^{-2})
-

➔ Réduction de deux ordres de grandeur

- SI Matrice parfaitement bandée
- ET Largeur de bande suffisamment

petite par rapport à la taille du cache

<relation>



list

(graph avec courbes qui apparaissent au bon moment)

CONCLUSION

On sélectionne une bande diagonale de la matrice, qu'on traite avec un cache set-associatif

- <relation>

Que faire du reste, hors de la bande ?

Granularité : Éléments isolés

- Incompatible avec les lignes de cache

➔ Pas de ligne de cache :

64o (8 mots) → 8o (1 mot)

Absence de structure claire

- Nombre de *hits* bas

➔ Pour augmenter la quantité de *hits*

dans un espace de cache modeste :

Cache complètement associatif

Transactions mémoires inadaptées

- Lectures et écritures sur 64o (8 mots)

➔ Transactions mémoires "par éléments"

RMW atomique sur 8o (1 mot)



(graph avec courbes qui apparaissent au bon moment)

CONCLUSION

On traite les éléments hors-bande de la matrice avec un cache « par élément », complètement associatif, avec des transactions mémoires « par élément » (RMW)

Conclusion

Deux régions pour traiter le dense et le creux

C'est ma contribution

Plan

1. Prérequis et Problématique

2. Approche 1 : Cache de réduction

3. Approche 2 : Partitionnement du cache

4. SpDCCache : Fonctionnement et Architecture Matérielle

5. Evaluations et Performances de SpDCCache

SpDCache

4 slides spdcache de ASAP
Quelques détails sur l'implem RTL

Plan

1. Prérequis et Problématique

2. Approche 1 : Cache de réduction

3. Approche 2 : Partitionnement du cache

4. SpDCache : Fonctionnement et Architecture Matérielle

5. Evaluations et Performances de SpDCache

RTL et python en meme temps

Traffic mémoire

Config 25/50/25

Puis accélération (Perf peu impressionnante à cause du setup minimaliste, mais une aubène pour ma thèse car permet d'analyser : on a des cas compute bound, ce qui n'arrive pas en pratique)

Répartition des matrices sur l'échelle dcd

Traffic mémoire

Set de 48 à 62 matrices

Toutes structures

Config 25/50/25

Perfs avec écart type (sous étiquettes)
conversion en "times"

Accélération

1 coeur CVA6, basse perf -> on est dans une zone entre compute et memory bound
Permet de clasifier les marices selon nouveau critere dcd
Explique les deux groupes d'accéléérations

Thanks !